42

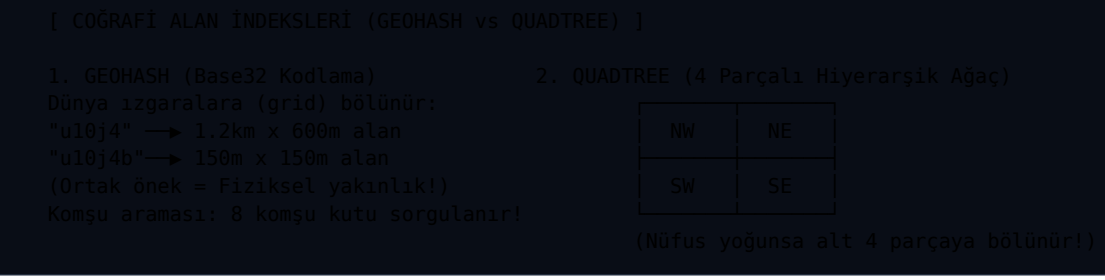
BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 1 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 1

Yakındaki Yerler / Mekan Arama Servisi (Proximity / Yelp): Geohash vs QuadTree vs Google S2

Designing a Proximity Service (Yelp / Google Maps): Geohash vs QuadTree

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Haritada 'Bana en yakın 10 kahveciyi bul' dediğinizde, dünyadaki 500 milyon kahvecinin koordinatını tek tek Pisagor teoremiyle ($x^2 + y^2$) hesaplamaya çalışırsanız sunucu patlar! Bunun yerine dünya bir satranç tahtası gibi karelere bölünür (Geohash). Her karenin posta kodu gibi bir adı vardır (ör. 'u10j4'). Siz o karenin içindeyseniz, sistem sadece o karenin ve etrafındaki 8 komşu karenin içindeki kahvecilere bakar. 500 milyon dükkan yerine sadece 50 dükkana bakarak 1 milisaniyede sonucu bulur!

🇬🇧 ENGLISH (ELI5)

Searching 'Find the 10 closest cafes near me' cannot compute the Pythagorean distance formula across all 500 million cafes worldwide on every GPS ping! Instead, divide the planet into an indexed grid of boxes (Geohash). Each grid box gets an alphanumeric postal code (e.g. `u10j4`). When you query from that box, the system simply queries that box plus its 8 immediate neighboring boxes. The search space shrinks from 500 million points down to 50 in 1 millisecond!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Seviyesi Mekan Arama (Proximity) Mimarisi: 1) Neden Standart SQL İndeksi Yetmez? Enlem (Latitude) ve Boylam (Longitude) iki boyutlu veridir; standart B-Tree indeksinde `WHERE lat BETWEEN ... AND lon BETWEEN ...` sorgusu indeks birleştirmesi (Index Merge) yüzünden çok yavaştır. 2) Geohash: Enlem ve boylamı iç içe geçirerek Base32 karakter dizisine kodlar (1D string). Özellik: Ortak önek (Prefix) paylaşan noktalar fiziksel olarak birbirine yakındır (Örn: `u10j4` 6 karakter ≈ 1.2 km). Kenar etkisi (Edge Effect) için merkez kutu + 8 komşu kutu sorgulanır. Redis `GEOADD` ve `GEORADIUS` komutları arka planda Geohash ve Sorted Set kullanır. 3) QuadTree: Her düğümün en fazla 4 çocuk düğümü olan hiyerarşik ağaçtır. Düğümdeki mekan sayısı eşiği (ör. 100 dükkan) aşınca o alan 4 alt parçaya bölünür (Yoğun şehir merkezlerinde derin, okyanusta sığ ağaç). 4) Google S2: Dünyayı küp projeksiyonu ve Hilbert Eğrisi ile 64-bit tam sayılara böler; Google Haritalar'ın standardıdır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Proximity Service Architecture: 1) The 2D Spatial Indexing Challenge: Traditional B-Trees index single dimensions efficiently; evaluating `WHERE lat BETWEEN x AND y AND lon BETWEEN a AND b` degrades to full table filtering. 2) Geohash: Interleaves binary bits of latitude and longitude into a Base32 string. Key Trait: Strings sharing prefixes share spatial proximity (`u10j4` has 1.2km precision). The Boundary Problem: Two adjacent points across a boundary can share completely different prefixes; solved by querying the central box plus its 8 surrounding neighbor cells. Redis native geospatial commands (`GEOADD`, `GEOSEARCH`) leverage 52-bit Geohashes within Sorted Sets. 3) QuadTree: In-memory 2D spatial tree where each quadrant recursively splits into 4 sub-quadrants when point density exceeds a threshold (e.g., 100 POIs). Deeply sub-divided in Manhattan; undivided across deserts. 4) Google S2 Geometry: Projects the spherical earth onto a cube using Space-Filling Hilbert Curves into 64-bit integers; de facto standard for Uber and Google Maps.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```java
// Java 25: Basitleştirilmiş Geohash Önek Eşleme ve Komşu Filtreleme
import java.util.*;

public class GeohashProximityEngine {
    public record Place(String name, String geohash6) {}

    private final Map<String, List<Place>> geohashIndex = new HashMap<>();

    public void indexPlace(Place place) {
        geohashIndex.computeIfAbsent(place.geohash6(), k -> new ArrayList<>()).add(place);
    }

    public List<Place> searchNearby(String userGeohash5Prefix) {
        List<Place> results = new ArrayList<>();
        // 5 karakterlik önek (~4.9 km x 4.9 km alan) ile başlayan tüm yerleri topla
        for (var entry : geohashIndex.entrySet()) {
            if (entry.getKey().startsWith(userGeohash5Prefix)) {
                results.addAll(entry.getValue());
            }
        }
        return results;
    }
}

void main() {
    var engine = new GeohashProximityEngine();
    engine.indexPlace(new GeohashProximityEngine.Place("Kadıköy Kahvecisi", "sxk97a"));
    engine.indexPlace(new GeohashProximityEngine.Place("Moda Dondurmacısı", "sxk97b"));
    engine.indexPlace(new GeohashProximityEngine.Place("Ankara Cafe", "sx5f2a"));

    IO.println("Kadıköy (sxk97) Civarındaki Mekanlar:");
    for (var p : engine.searchNearby("sxk97")) {
        IO.println("• " + p.name() + " [Geohash: " + p.geohash6() + "]");
    }
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta mekan bilgilerinin (isim, adres, menü) QuadTree veya Redis'in içinde saklanacağını söylemeyin! Mekan verisi (metadata) PostgreSQL/MySQL'de tutulur; QuadTree veya Geohash indeksinde SADECE `Place_ID` ve `Geohash` tutulmalıdır. Aksi halde önbellek belleği çöker.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Do not pack heavy business payloads (reviews, menu photos, descriptions) inside the spatial QuadTree/Geohash index. The spatial index must hold only compact `(Place_ID, Coordinates)` tuples; full metadata is resolved from the primary DB.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Geohash 2D koordinatları tek boyutlu string'lere dönüştürerek hızlı arama sağlar.

• Kenar etkisi (Boundary Effect) problemini çözmek için merkez hücre + 8 komşu taranır.

• QuadTree bellek içi yoğunluğa göre dinamik olarak alt parçalara bölünür.

🎯 Key Takeaways for the Interview

• Geohash flattens 2D coordinates into 1D strings, enabling standard index range queries.

• Mitigate Geohash boundary cliffs by querying the central cell plus its 8 adjacent neighbors.

• QuadTrees adapt dynamically to population density, allocating deep nodes only in dense cities.